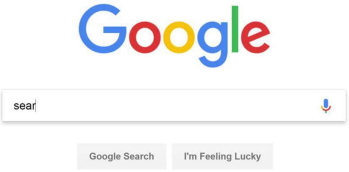
Introduction to WWW, Web Protocols and URLs
Common Terms

- Internet vs. Web
- Web Browsers
- URL
- Web Server
- DNS
- HTTP Protocol
- HTTPS

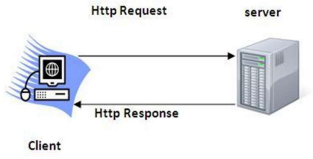
Introduction to WWW, Web Protocols and URLs
Internet vs. WWW

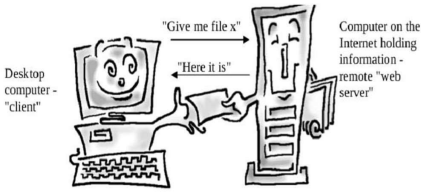


Introduction to WWW, Web Protocols and URLs
How does WWW work?

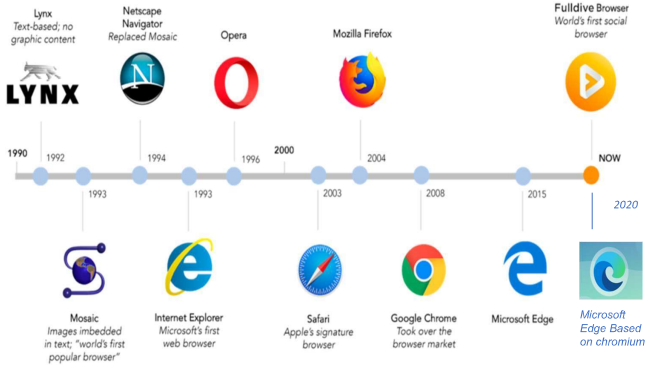
1. Client/Server Architecture

2. Request/Response Pattern





Introduction to WWW, Web Protocols and URLs
History of Web Browsers

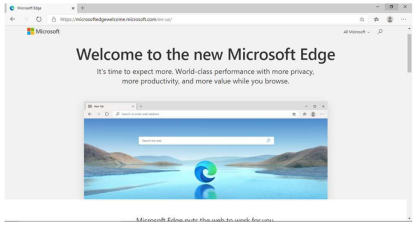


Source:A Brief History of Browsers https://medium.com/@fulldive/a-brief-history-of-browsers-57669527c0cf

Introduction to WWW, Web Protocols and URLs
Browser Evolution

Lynx – A text based browser

Mosaic – the first graphical browser



Source: Browser Museum
http://www.donmouth.co.uk/web_design/browsermuseum/browsermuseum.html

Introduction to WWW, Web Protocols and URLs
URLs

- URL stands for Uniform Resource Locator
- General form:

  scheme:object-address

- For the http protocol, the object-address is:

  fully qualified domain name/doc path

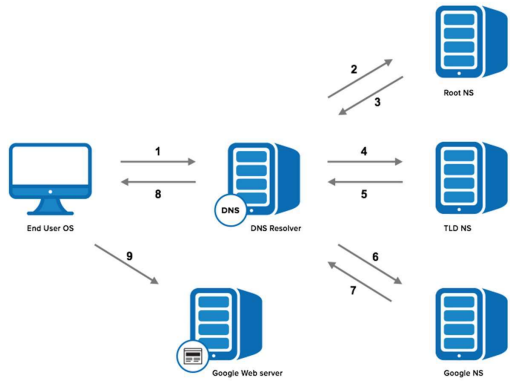
Example:

https://www.amazon.com/international-sales-offers.html

Introduction to WWW, Web Protocols and URLs
Web Servers

- General Web Server Characteristics
  - Web servers have two main directories:
    1. Document root (servable documents)
    2. Server root (server system software)
  - Document root is accessed indirectly by clients
    - Its actual location is set by the server configuration file
    - Requests are mapped to the actual location
- Popular Examples
  - Apache
  - IIS

Introduction to WWW, Web Protocols and URLs
Domain Name Service



Step 1: OS Recursive Query to DNS Resolver
Step 2: DNS Resolver Iterative Query to the Root Server
Step 3: Root Server Response
Step 4: DNS Resolver Iterative Query to the TLD Server
Step 5: TLD Server Response
Step 6: DNS Resolver Iterative Query to the Google.com NS
Step 7: Google.com NS Response
Step 8: DNS Resolver Response to OS
Step 9: Browser Starts TCP Handshake

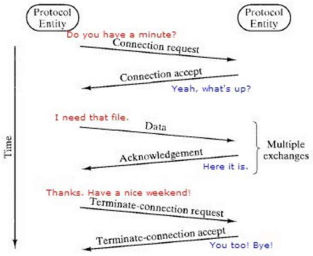
Introduction to WWW, Web Protocols and URLs
How to get your own website?

Steps:

1. Choose a domain name
2. Register a domain and sign up with web hosting
3. Set up a website using WordPress/Name cheap/Go Daddy (through web host)
4. Customize your website design and structure
5. Add pages and content to your website

Introduction to Web Protocols and HTTP
What is a Protocol?

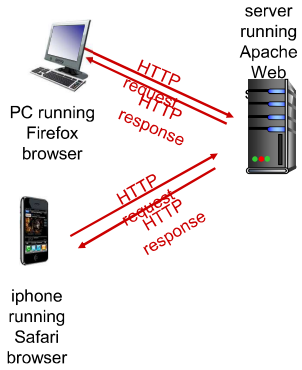
- A protocol is a set of rules and guidelines for communicating data.
- Different applications use different protocols
- The web, in particular, uses multiple protocols to communicate.
- The most important and visible protocols are HTTP and HTTPS.



Introduction to Web Protocols and HTTP
HTTP Overview

HTTP: HyperText Transfer Protocol

- Application Protocol used by the Web
- Client/Server model
  - *Client:* browser that requests, receives, and "displays" Web Objects
  - *Server:* Web server sends Web Objects (using HTTP protocol) in response to requests



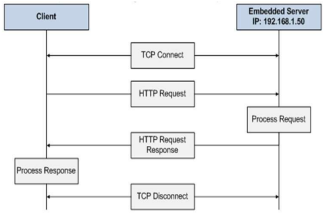
Introduction to Web Protocols and HTTP
HTTP Overview…(cntd.)

uses TCP:

- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

HTTP is "stateless"

- server maintains no information about past client requests



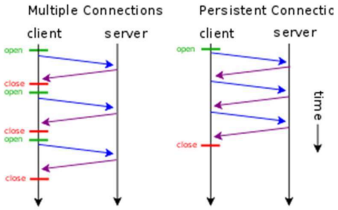
Introduction to Web Protocols and HTTP
HTTP Connections

non-persistent HTTP

- at most one object sent over TCP connection
  - connection is then closed
- downloading multiple objects required multiple connections

persistent HTTP

- multiple objects can be sent over single TCP connection between client, server



Introduction to Web Protocols and HTTP
HTTP Requests

- HTTP request is a *request line*, followed by zero or more *request headers*
- Request line: <method> <uri> <version>
  - <version> is HTTP version of request (HTTP/1.0 or HTTP/1.1)
  - <uri> is typically URL for proxies, URL suffix for servers.
  - <method> is either GET, POST, OPTIONS, HEAD, PUT, DELETE, or TRACE.
- Request Header
- Blank line (CRLF)
- Message Body

```
GET /test.html HTTP/1.1
Accept: */*
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 4.01;
Windows 98)
Host: euro.ecom.cmu.edu
Connection: Keep-Alive
CRLF (\r\n)
```

Introduction to Web Protocols and HTTP
HTTP Request Methods

- HTTP methods:
  - GET: Retrieve static or dynamic content
  - POST: Send content to server through request body
  - OPTIONS: Get server or file attributes
  - HEAD: Fetches only header field without any response body
  - PUT: Write a file to the server
  - DELETE: Delete a file on the server

Introduction to Web Protocols and HTTP
HTTP Response

- HTTP response is a response line followed by zero or more response headers.
- Response line:
- <version> <status code> <status msg>
  - <version> is HTTP version of the response.
  - <status code> is numeric status.
- Response headers:

  <header name>: <header data>
  - Provide additional information about response
  - Content-Type: MIME type of content in response body.
  - Content-Length: Length of content in response body.

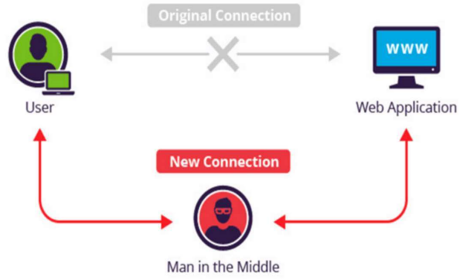
```
HTTP/1.1 200 OK
Date: Thu, 22 Jul 1999 04:02:15 GMT
Server: Apache/1.3.3 Ben-SSL/1.28 (Unix)
Last-Modified: Thu, 22 Jul 1999 03:33:21 GMT
ETag: "48bb2-4f-37969101"
Accept-Ranges: bytes
Content-Length: 79
Keep-Alive: timeout=15, max=100
Connection: Keep-Alive
Content-Type: text/html
CRLF
<html>
<head><title>Test page</title></head>
<body>
<h1>Test page</h1>
</html>
```

HTTP Response : Status Codes

- Three-digit number; first digit specifies the general status
  - 1 => Informational
  - 2 => Success
  - 3 => Redirection
  - 4 => Client error
  - 5 => Server error
- <status msg> is corresponding English text.
  - 200 OK => Request was handled without error
  - 403 Forbidden => Client lacks permission to access file
  - 404 Not found => Server couldn't find the file.

HTTP Secure (HTTPS)

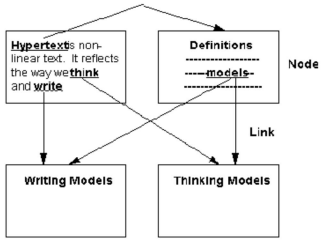
- A common security attack
- Need to encrypt data to save it from such attacks



HTML – Basic Markups
Introduction

HTML– Hyper Text Markup Language

Hypertext - cross-referencing /linking between related sections of text and associated graphic material



Hyper Text-Text which contains links to other text, web pages, images, files audio, video

Markup - define elements within a document using tags

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE recipe PUBLIC "-//Happy-Monkey//DTD RecipeBook//EN"
"http://www.happy-monkey.net/recipebook/recipebook.dtd">

<recipe>

    <title>Peanut-butter On A Spoon</title>

    <ingredientlist>
        <ingredient>Peanut-butter</ingredient>
    </ingredientlist>

    <preparation>
        Stick a spoon in a jar of peanut-butter,
        scoop and pull out a big glob of peanut-butter.
    </preparation>

</recipe>
```

HTML – Basic Markups
Basic Syntax

- Elements are defined by tags (markers)
  - Tag format:
    - Opening tag: **<tag_name>**
    - Closing tag: **</tag_name>**

    Syntax:

    <tag_name>
    Content…
    </tag_name>
- Not all tags have content
  - If a tag has no content, its form is **<tag_name … />**
- The container and its content together are called an *element*

HTML – Basic Markups
Comments

- An HTML comment begins with **<! – –** and the comment closes with **– –>.**
- HTML comments are visible to anyone that views the page source code, but are not rendered when the HTML document is rendered by a browser.

```
You will be able to see this text.

<!-- You will not be able to see this text. -->

You can even comment out things in <!-- the middle of --> a sentence.

<!--
Or you can
comment out
a large number of lines.
-->

<div class="example-class">
Another thing you can do is put comments after closing tags, to help you find
where a particular element ends. <br>
(This can be helpful if you have a lot of nested elements.)
</div> <!-- /.example-class -->
```

HTML – Basic Markups
Document Structure

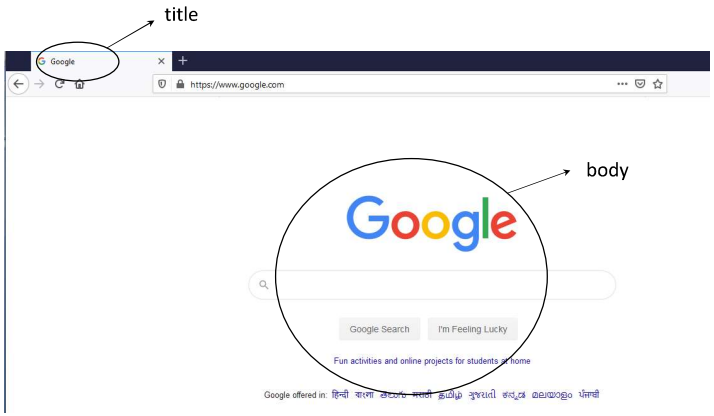
```
<html>
  <head>
      <title>… </title>
  </head>
  <body>
      …
  </body>
</html>
```

HTML – Basic Markups
Document Structure Elements

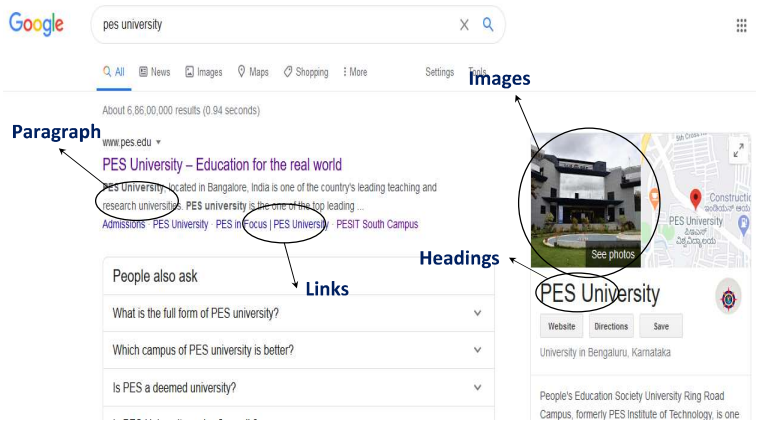
The following elements are part of every web page.

Element	Description
<html></html>	Surrounds the entire page
<head></head>	Contains header information (metadata, CSS styles, JavaScript code)
<title></title>	Holds the page title normally displayed in the title bar and used in search results
<body></body>	Contains the main body text. All parts of the page normally visible are in the body

HTML – Basic Markups
Document Structure



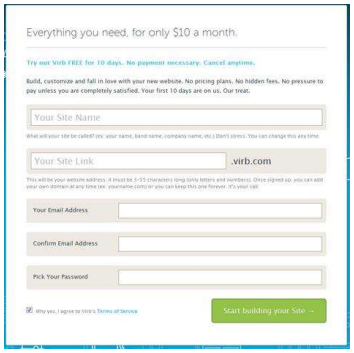
HTML – Basic Markups
Common Tags / Elements

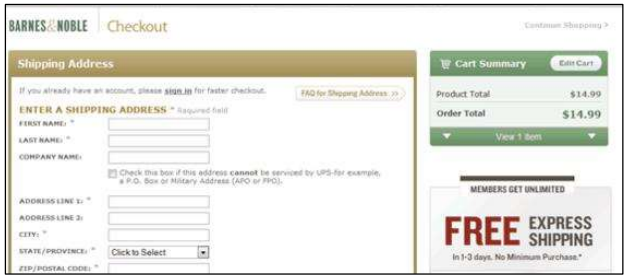


HTML – Basic Markups
Common Tags / Elements

- Text Content
  - Paragraphs
  - Headings
  - Lists
  - Tables
- Images
- Links

HTML – Forms
Introduction





HTML – Forms
Introduction

- An HTML form is used to collect user input.
- A form is a way to send information from a browser to a server
- All the components of a form appear as the content of <form> tag
  - The components are called *widgets* (e.g., text boxes, radio buttons and checkboxes)

- A simple form may look like this:

First name:
John
Last name:
Doe
Submit

HTML – Forms
Attributes

Syntax:

```
<form list_of_attributes_and_values>
    <input element>
    ...
</form>
```

Important attributes of the <form> tag

- Method
- Action
- Target

Example:

```
<form method="post" action="survey.php" target="_blank">
    <input type="text">
    ...
</form>
```

HTML – Forms
Methods

GET:

- Appends the form data to the URL, in name/value pairs
- NEVER use GET to send sensitive data! (the submitted form data is visible in the URL!)
- The length of a URL is limited (2048 characters)
- Useful for form submissions where a user wants to bookmark the result
- GET is good for non-secure data, like query strings in Google

POST:

- Appends the form data inside the body of the HTTP request (the submitted form data is not shown in the URL)
- POST has no size limitations, and can be used to send large amounts of data.
- Form submissions with POST cannot be bookmarked

HTML – Forms
Input widgets

- Input widget can be any of the following types
  - Text
  - Textarea
  - Button
  - Checkboxes
  - Radio Buttons
  - Dropdown list
  - Hidden

HTML – Forms
Input element

- The HTML <input> element is the most used form element.
- An <input> element can be displayed in many ways, depending on the type attribute.

Type	Description
<input type="text">	Displays a single-line text input field
<input type="radio">	Displays a radio button (for selecting one of many choices)
<input type="checkbox">	Displays a checkbox (for selecting zero or more of many choices)
<input type="submit">	Displays a submit button (for submitting the form)
<input type="button">	Displays a clickable button

```
<label for="fname" >First Name:
            <input type="text" name="name"
id="fname"> <br><br>
            </label>
```

The <fieldset> tag is used to group related elements in a form.
The <fieldset> tag draws a box around the related elements.

HTML – Forms
HTML5 – New Input widgets

- HTML5 specifications introduced new Input types
  - email : email address
  - number: spinbox
  - range: slider
  - url: web addresses
  - color: color pickers
  - search: search boxes
  - date: date
  - month: month
  - time: time
  - week: week
  - datetime: combination of date and time.

HTML5: Input - e-mail

- The email type is used for input fields that should contain an e-mail address.
- The value of the email field is automatically validated when the form is submitted.

E-mail: <input type="email" name="user_email" />

HTML5: Input - url

- The url type is used for input fields that should contain a URL address.
- The value of the url field is automatically validated when the form is submitted.

Homepage: <input type="url" name="user_url" />

HTML5: Input - number

- The number type is used for input fields that should contain a numeric value.
- Set restrictions on what numbers are accepted:

Points: <input type="number" name="points" min="1" max="10" />

Attribute	Value	Description
max	*number*	Specifies the maximum value allowed
min	*number*	Specifies the minimum value allowed
step	*number*	Specifies legal number intervals (if step="3", legal numbers could be -3,0,3,6, etc)
value	*number*	Specifies the default value

HTML5: Input - range

- The range type is used for input fields that should contain a value from a range of numbers.
- The range type is displayed as a slider bar.
- You can also set restrictions on what numbers are accepted:

<input type="range" name="points" min="1" max="10" />

HTML5: Input – date pickers

HTML5 has several new input types for selecting date and time:

> date - Selects date, month and year
> month - Selects month and year
> week - Selects week and year
> time - Selects time (hour and minute)
> datetime - Selects time, date, month and year. This is now obsolete
> datetime-local - Selects time, date, month and year (local time)

HTML5: Input – color picker

- The color type is used for input fields that should contain a color.
- This input type will allow you to select a color from a color picker:

Color: <input type="color" name="user_color" />

HTML5
Introduction

- HTML5 ~= HTML + CSS3 + JavaScript API
- Future of the web
- It comes with new tags, features and APIs
- HTML5 introduces 28 new elements such as:
  <header>,<footer>,<article>,<nav>,<section>,<time>,<audio>,<video>,<output> etc..
- The main structural elements are as follows:

Element	Description
Article	Defines an article (for example within a section)
Footer	Footer elements contain information about their containing element: who wrote it, copyright, etc.
Header	The page header shown on the page, not the same as <head>
Nav	Collection of links to other pages
Section	A part or chapter in a book, or essentially the content body of the page

HTML5
Features

HTML 5
Forms – New Input Widgets

- HTML5 specifications introduced new Input types and properties

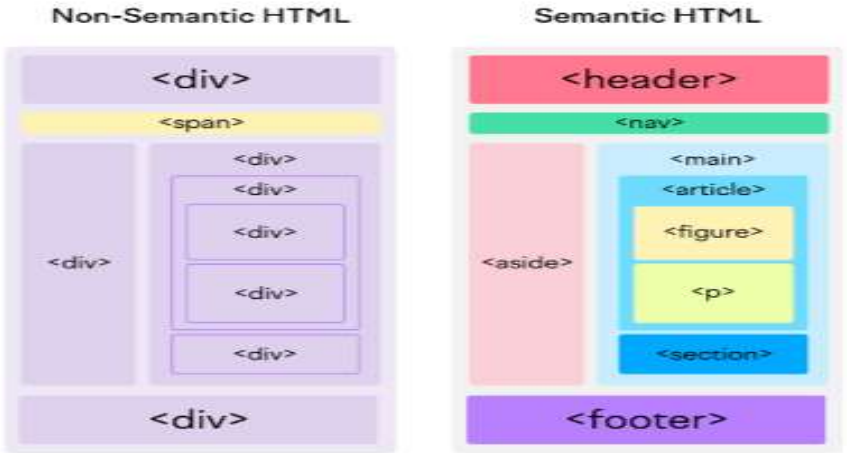
New Input Types:

- email: email address
- number: spinbox
- range: slider
- url: web addresses
- color: color pickers
- search: search boxes
- date: date
- time: time
- file: input file selection
- Tel: phone no.

New Input properties / Attributes:

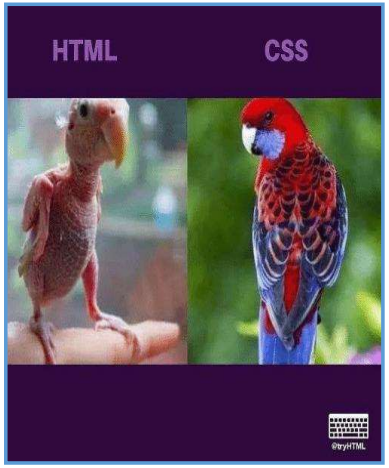
Attribute	Description
Min, Max	Accepted min and max values
Multiple	Related to file input type, allows selection of multiple files
Pattern	Specifies a pattern used to validate an input field
Placeholder	A short hint intended to aid the user with data entry
Required	Boolean attribute to indicate that the element is required
Step	Limits allowed values, thus indicating the granularity required

HTML 5
Semantic Elements



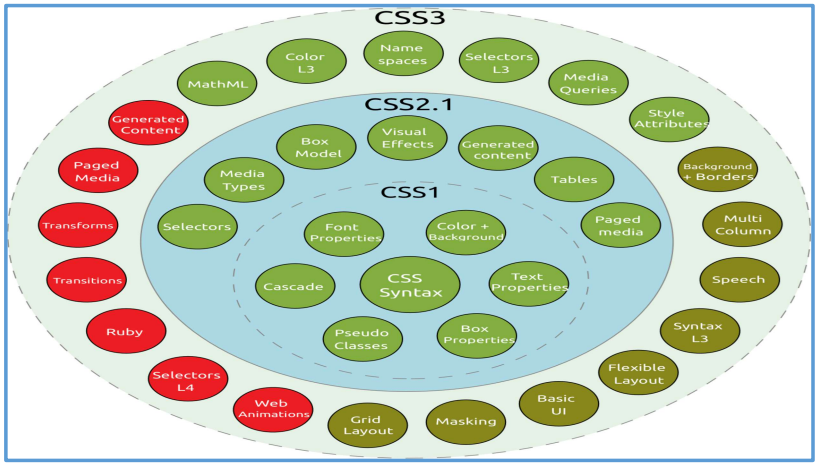
CSS – Cascading Style Sheets and Selectors

Introduction



CSS – Cascading Style Sheets and Selectors

Introduction



CSS – Cascading Style Sheets and Selectors

Three ways to include CSS

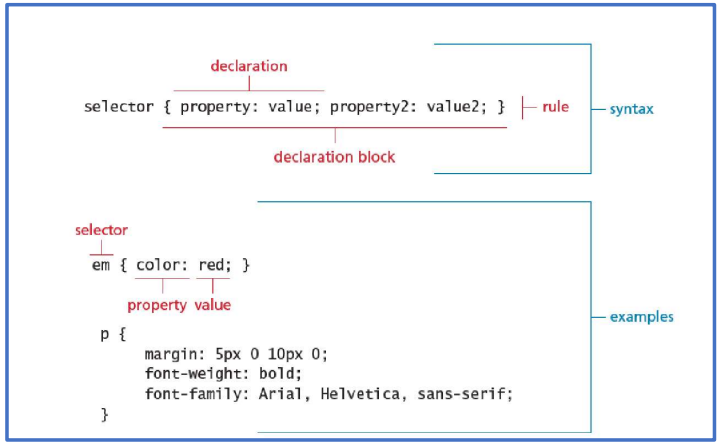
1) Inline Style - CSS is placed directly into the HTML element.

1) Internal Style Sheet /Embedded Style Sheet - CSS is placed into a separate area within the <head> section of a web page using <style> tag.

1) External Style Sheet - CSS is placed into a separate file and "connected" to a web page.

CSS – Cascading Style Sheets and Selectors

Syntax



CSS – Cascading Style Sheets and Selectors

Syntax

- Every CSS rule begins with a **selector**.
- The selector identifies which element or elements in the HTML document will be affected by the declarations in the rule

CSS – Cascading Style Sheets and Selectors

Properties

Property Type	Property
Fonts	font font-family font-size font-style font-weight @font-face
Text	letter-spacing line-height text-align text-decoration text-indent
Color and background	background background-color background-image background-position background-repeat color
Borders	border border-color border-width border-style border-top border-top-color border-top-width

CSS – Cascading Style Sheets and Selectors

Properties

Property Type	Property
Spacing	padding padding-bottom, padding-left, padding-right, padding-top margin margin-bottom, margin-left, margin-right, margin-top
Sizing	height max-height max-width min-height min-width width
Layout	bottom, left, right, top clear display float overflow position visibility z-index
Lists	list-style list-style-image list-style-type

CSS – Cascading Style Sheets and Selectors

Properties

- Some property values are from a predefined list of keywords.
- Others are values such as length measurements, percentages, numbers without units, color values, and URLs.

CSS – Cascading Style Sheets and Selectors

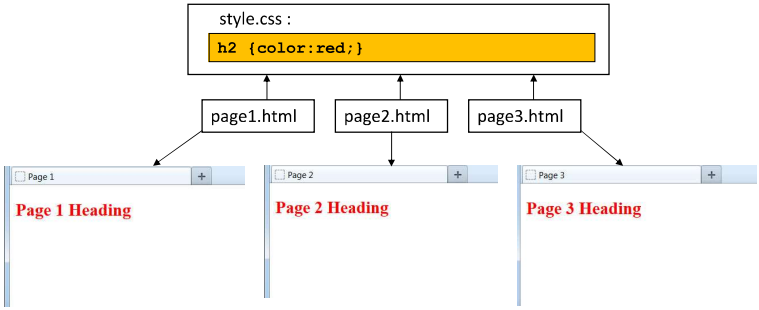
Color values in CSS

Method	Description	Example
Name	Use one of 17 standard color names. CSS3 has 140 standard names.	`color: red;` `color: hotpink; /* CSS3 only */`
RGB	Uses three different numbers between 0 and 255 to describe the red, green, and blue values of the color.	`color: rgb(255,0,0);` `color: rgb(255,105,180);`
Hexadecimal	Uses a six-digit hexadecimal number to describe the red, green, and blue value of the color; each of the three RGB values is between 0 and FF (which is 255 in decimal). Notice that the hexadecimal number is preceded by a hash or pound symbol (#).	`color: #FF0000;` `color: #FF69B4;`
RGBa	This defines a partially transparent background color. The "a" stands for "alpha", which is a term used to identify a transparency that is a value between 0.0 (fully transparent) and 1.0 (fully opaque).	`color: rgb(255,0,0, 0.5);`
HSL	Allows you to specify a color using Hue Saturation and Light values. This is available only in CSS3. HSLA is also available as well.	`color: hsl(0,100%,100%);` `color: hsl(330,59%,100%);`

CSS – Cascading Style Sheets and Selectors

Benefits of External Style Sheets

The real power of using an external style sheet is that multiple web pages on our site can link to the same style sheet:



Styles declared in an external style sheet will affect all matching elements on all web pages that link to the style sheet. By editing the external style sheet, we can make site-wide changes (even to hundreds of pages) instantly.

CSS – Cascading Style Sheets and Selectors

Internal vs. External Style Sheets

Internal Style Sheets

- are appropriate for very small sites, especially those that have just a single page.
- might also make sense when each page of a site needs to have a completely different look.

External Style Sheets:

- are better for multi-page websites that need to have a uniform look and feel to all pages.
- make for faster-loading sites (less redundant code).
- allow designers to make site-wide changes quickly and easily.

External style sheets create the furthest separation between content and presentation. Hence, the best option when creating a new site.

CSS – Cascading Style Sheets and Selectors

Conflicts

- Same formatting rules can be defined in all three locations at the same time.
- For example, a paragraph element could contain an inline style (color:red) but the internal style sheet (color:blue) and the external style sheet (color:green) give conflicting instructions to the web browser.
- Web browsers need a consistent way of "settling" this disagreement.

Cascading in CSS

- We use the term cascading because there is an established order of priority to resolve these formatting conflicts:
  1) Inline style (highest priority)
  2) Internal style sheet (second priority)
  3) External style sheet (third priority)
  4) Web browser default (only if not defined elsewhere)

For each XHTML element, the browser will combine all the styles defined at different levels. For all conflicts, it will use the above priority system to determine which format to display on the page.

In the prior example, the paragraph would display as red, because the inline style "outranks" all the others.

{CSS}
SELECTORS

CSS Selectors

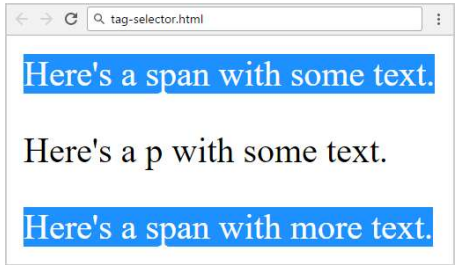
Select the Elements to Apply CSS Rules

Selectors

- Primary Selectors
  - Element selector
  - Class Selectors
  - ID selectors
- Nested Selector
- Multiple Selector
- Pseudo-selector
- Attribute selector

Primary Selectors: Select by Tag – ELEMENT SELECTORS

```
<span>Here's a span with some text.</span>
  <p>Here's a p with some text.</p>
<span>Here's a span with more text.</span>
```



Select all elements

```
span {
  background: DodgerBlue;
  color: #ffffff;
}
```

Primary Selectors: Select by ID

```
<span id="top">Here's a span with some text.</span>
<span>Here's another.</span>
```

Select with id="top"

```
span#top {
  background: DodgerBlue;
}
```


Primary Selectors: Select by Class

The *.class* selector selects elements with a specific class attribute.

To select elements with a specific class, write a period (.) character, followed by the name of the class.

Primary Selectors: Select by Class

```
<span class="sky">Here's a span with some text.</span>
<span>
  Another <span class="code">&lt;span&gt;</span>.
</span>
```

 with class="sky"

```
span.sky {
  background: DodgerBlue;
}
.code {
  font-family:
    Consolas;
}
```

Elements with class="code"

Pseudo-class selector

- **What are Pseudo-classes?**
- A pseudo-class is used to define a special state of an element.

For example, it can be used to:

- Style an element when a user hovers over it
- Style visited and unvisited links differently
- Style an element when it gets focus.

Syntax:

selector:pseudo-class { property:value; }

- The **order** of these pseudo-class elements is important.
- **The :Link, :Visited, :Focus, :Hover-** Order of pseudo-classes.

Pseudo-Element selector

- A CSS pseudo-element is used to style specified parts of an element.

For example, it can be used to:

- Style the first letter, or line, of an element
- Insert content before, or after, the content of an element
- Syntax:

selector:pseudo-element { property:value; }

Common Pseudo-element & Pseudo-class selectors

a:link	pseudo-class	Selects links that have not been visited
a:visited	pseudo-class	Selects links that have been visited
:focus	pseudo-class	Selects elements (such as text boxes or list boxes) that have the input focus.
:hover	pseudo-class	Selects elements that the mouse pointer is currently above.
:active	pseudo-class	Selects an element that is being activated by the user. A typical example is a link that is being clicked.
:checked	pseudo-class	Selects a form element that is currently checked. A typical example might be a radio button or a check box.

Common Pseudo-element & Pseudo-class selectors

:first-child	pseudo-class	Selects an element that is the first child of its parent. A common use is to provide different styling to the first element in a list.
:first-letter	pseudo-element	Selects the first letter of an element. Useful for adding drop-caps to a paragraph.
:first-line	pseudo-element	Selects the first line of an element.

Attribute Selectors

Selector	Matches	Example
[]	A specific attribute.	`[title]` Matches any element with a title attribute
[=]	A specific attribute with a specific value.	`a[title="posts from this country"]` Matches any <a> element whose title attribute is exactly "posts from this country"
[~=]	A specific attribute whose value matches at least one of the words in a space-delimited list of words.	`[title~="Countries"]` Matches any title attribute that contains the word "Countries"
[^=]	A specific attribute whose value begins with a specified value.	`a[href^="mailto"]` Matches any <a> element whose href attribute begins with "mailto"
[*=]	A specific attribute whose value contains a substring.	`img[src*="flag"]` Matches any <img> element whose src attribute contains somewhere within it the text "flag"
[$=]	A specific attribute whose value ends with a specified value.	`a[href$=".pdf"]` Matches any <a> element whose href attribute ends with the text ".pdf"

Nested Selectors: Descendant

```
div.items a {
  color: green;
  font-weight: bold;
}
```

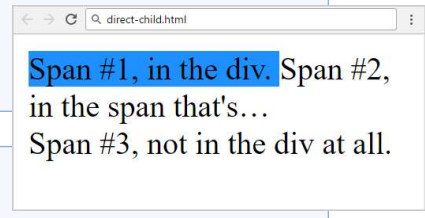
<a> inside


```
<div class="items">
  <a href="#">Item1</a>
  <a href="#">Item2</a>
  <a href="#">Item3</a>
  <ul>
    <li><a href="#">Item4</a></li>
    <li><a href="#">Item5</a></li>
    <li><a href="#">Item6</a></li>
  </ul>
</div>
```

Nested Selectors: Direct Child

```
div > span {
  background: DodgerBlue;
}
span { background: #fff; }
```

 directly contained in a



Direct child of

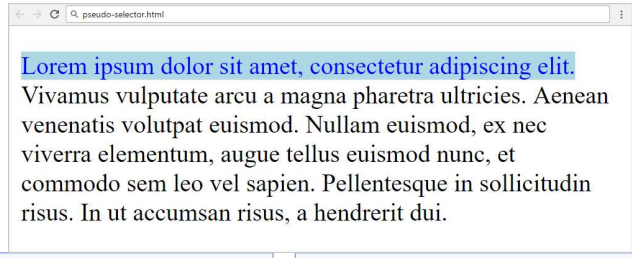
```
<div>
  <span>Span #1, in the div.
    <span>Span #2, in the span that's…</span>
  </span>
</div>
<span>Span #3, not in the div at all.</span>
```

Multiple Selectors (Element, Element)


```
<h1>Welcome…</h1>
<h2>My name is…</h2>
<p>I live in Duckburg.</p>
<p>My best friend is…</p>
```

```
h1, h2, p {
  background: yellow;
}
```

Pseudo Selector (:first-line)



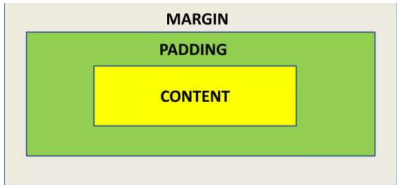

```
<p>Lorem ipsum dolor
sit amet, consectetur
adipiscing elit.
</p>
```

```
p:first-line {
  color:blue;
  background-color: lightblue;
}
```

CSS – Box Model and Position Property

Introduction

- All HTML elements can be considered as boxes.
- The term “box model” is used when talking about design and layout
- CSS box model is essentially a box that wraps around every HTML element.
- It consists of:
  - ❑Margins
  - ❑Borders
  - ❑Padding
  - ❑Actual Content



- It allows us to add a border around elements and define space between elements.

CSS – Box Model and Position Property

Element support for styling

- There are two elements used commonly to style specific parts of a webpage
  -
    - To apply style to a part of a paragraph
    - In-line element
  -
    - To apply style to a set of elements or paragraphs
    - Block element

CSS – Box Model and Position Property

Element support for styling

Block-level elements

- A block-level element always starts on a new line, and the browsers automatically add some space (a margin) before and after the element.
- A block-level element always takes up the full width available (stretches out to the left and right as far as it can).

Some example block-level elements are:

• , <h1>-<h6>, <p>, <ul>,<nav>

In-Line elements

- An inline element does not start on a new line.
- An inline element only takes up as much width as necessary.
- Some example inline-level elements are:

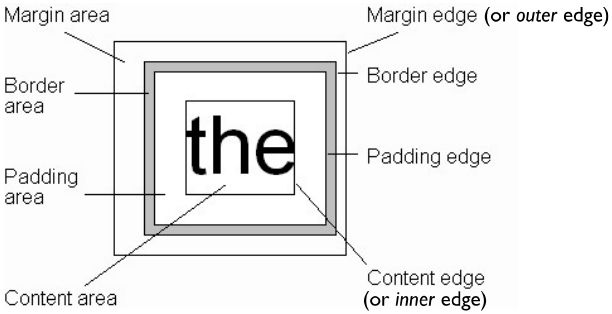
•, <a>, <input>,<button>,<img>

•Block element cannot be nested inside an inline element;

CSS – Box Model and Position Property

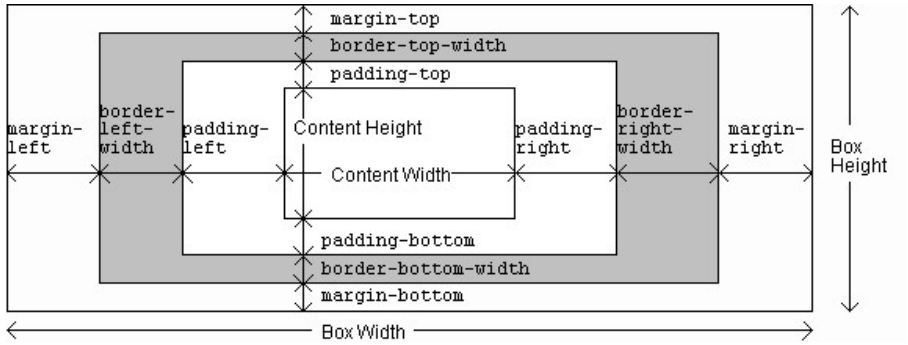
Box Model

- Every rendered element occupies a box:



CSS – Box Model and Position Property

Box Model - Components



CSS – Box Model and Position Property

Position Property

- By default, the browser determines the positioning of each element
- CSS introduced the **position** property and a capability to control how and where page elements are displayed
- Position property values:
  - **Static:** default value. Elements are positioned in the normal flow of the document
  - **Absolute** -relative to the nearest positioned ancestor. Removed from normal flow
  - **Relative -** is positioned relative to its normal position.
  - **Fixed -**it always stays in the same place.
  - **Sticky-** toggles between relative and fixed, positioned relative until a given offset position is met in the viewport - then it "sticks" in place.

CSS – Box Model and Position Property

Element Width and Height

- The total width of an element is calculated as:
- **Total element width** = width + left padding + right padding + left border + right border + left margin + right margin

- The total height of an element is calculated as:
- **Total element height** = height + top padding + bottom padding + top border + bottom border + top margin + bottom margin

- Example: This element will have a total width of 350px:

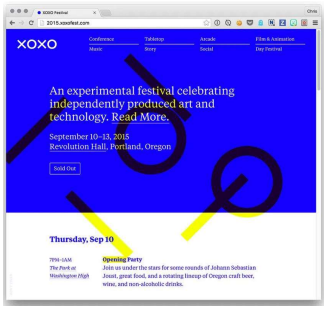
```
div {
  width: 320px;
  padding: 10px;
  border: 5px solid gray;
  margin: 0;
}
```

Calculation:
320px (width)
+ 20px (left + right padding)
+ 10px (left + right border)
+ 0px (left + right margin)
= 350px

CSS – Box Model and Position Property

Background

- CSS provides control over the backgrounds of block-level elements.
- CSS can set a background color or add background images to HTML5 elements.
- Different properties
  - background-image
  - background-position
  - background-repeat
  - background-attachment



- CSS has several different units to represent lengths.
- Most CSS properties take length values, such as **width**, **margin**, **padding**, **font-size**, etc.
- Each length value is followed by a length **unit**.
- **Negative lengths** are allowed only for some CSS properties only.

CSS – Box Model and Position Property
Length Units

Absolute length units

These are fixed and the length expressed in this format will appear of the same size

Unit	Description	Calculation
cm	centimeters	1cm = 96px/2.54
mm	millimeters	1mm = 1/10th of 1cm.
in	inches	1in = 2.54cm = 96px
px	pixels	1px = 1/96th of 1in.
pt	points	1pt = 1/72nd of 1in
pc	picas	1pc = 12pt = 1/6th of 1in

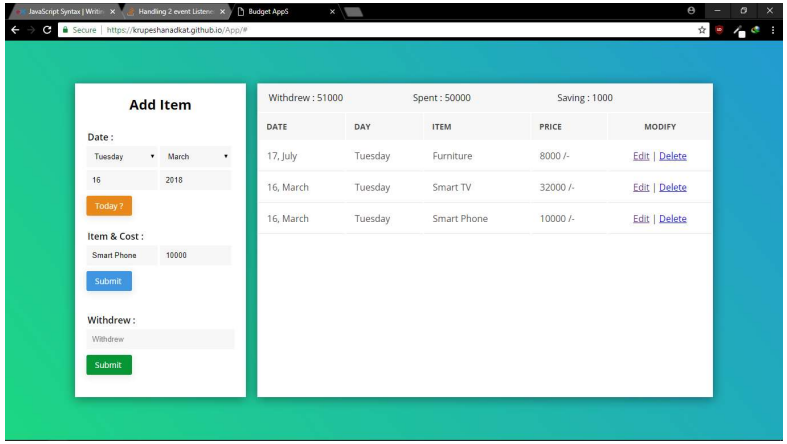
CSS – Box Model and Position Property
Length Units

Relative length units

Relative length expressed in this format will appear relative to other reference elements

Unit	Description
em	Relative to the font-size of the element (2em is 2 times current font)
ex	Relative to the x-height of current font
ch	Relative to width of the “0” (zero)
rem	Relative to the font-size of the root element
%	Relative to the parent element

JavaScript - Basics
Introduction to JavaScript



JavaScript - Basics
Introduction to JavaScript

- Client Side Scripting Language
- Originally, LiveScript in NetScape Browser
- JavaScript programs are run by an interpreter built into the user's web browser
- Now the language has evolved with additional Server Side Scripting capabilities (like in Node.JS)

JavaScript - Basics
Introduction to JavaScript...(cntd.)

- Client renders (displays) the response received from server (mix of HTML and JavaScript)
- Browser displays HTML
- And runs any JavaScript code within the HTML
- dynamic programming language that can add interactivity to a website.

JavaScript - Basics
Introduction to JavaScript...(cntd.)

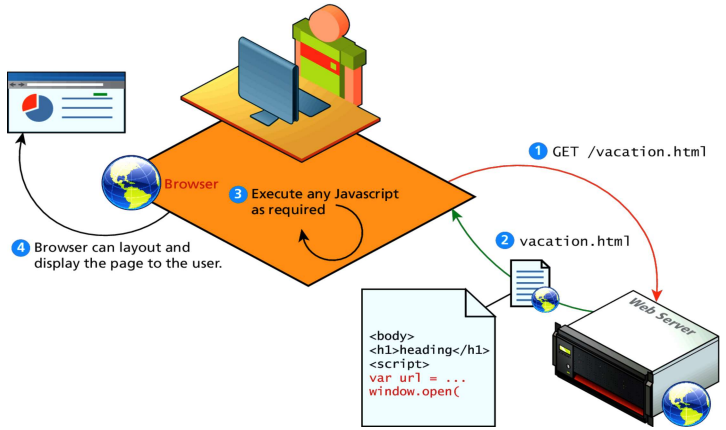
Pros and Cons of JavaScript

- Pros
  - Allows more dynamic HTML pages, even complete web applications
- Cons
  - Requires a JavaScript-enabled browser
  - Requires a client who trusts the server enough to run the code the server provides
- JavaScript has some protection in place but can still cause security problems for clients

1. **HTML** to define the content of web pages
2. **CSS** to specify the layout of web pages and give a good look.
3. **JavaScript** to program the behaviour of web pages

- Web pages are not the only place where JavaScript is used. Many desktop and server programs use JavaScript.
- Node.js is the best known.
- Some databases, like MongoDB and CouchDB, also use JavaScript as their programming language.

Client side scripting



JavaScript Code

JavaScript can be inserted into documents by using the <SCRIPT> tag

```
<html>
        <head>
                <title>Hello World in JavaScript</title>
        </head>
        <body>
                <script type="text/javascript">
                        document.write("Hello World!");
                </script>
        </body>
</html>
```


JavaScript Code

Where to Put your Scripts

- You can have any number of script tags at any position
- Scripts can be placed in the HEAD or in the BODY
  - In the HEAD, scripts are run before the page is displayed
  - In the BODY, scripts are run as the page is displayed
  - In the HEAD is the right place to define functions and variables that are used by scripts within the BODY

JavaScript - Basics
JavaScript Code – External Scripts

- Scripts can also be loaded from an external file
- This is useful if you have a complicated script or set of subroutines that are used in several different documents

```
<script src="myscript.js"></script>
```

- The myscript.js should not include the script tags

JavaScript - Basics
JavaScript Code - Debugging

Debugging JavaScript Errors

- When you're learning or using JavaScript, it's important to be able to track typing or other coding errors.

Browser	How to access JavaScript error messages
Apple Safari	Select Safari → Preferences → Advanced → "Show Developer menu in menu bar." You may prefer to use the Firebug Lite JavaScript module, which many people find easier to use.
Google Chrome	Press Ctrl-Shift-J on a PC, or Command-Shift-J on a Mac.
Mozilla Firefox	Press Ctrl-Shift-J on a PC, or Command-Shift-J on a Mac.
Microsoft Internet Explorer & Edge	Press F12 to call up the DevTools Console.
Opera	Select Tools → Advanced → Error Console.

JavaScript - Basics
JavaScript Code - Comments

Comments in JavaScript

- Single line comment : //
- Multiline comments : /* */

JavaScript - Basics
JavaScript Code – Using Semicolon

- JavaScript generally automatically inserts semicolons at the end of line

  x += 10 => x += 10;

- However, when you wish to place more than one statement on a line, you must separate them with semicolons, like this:

  x += 10; y -= 5; z = 0

- When a statement spans across multiple lines, JavaScript will not raise error if the next line has a valid symbol/literal/token

  return a

  + b

JavaScript - Basics
JavaScript Code – Variable Declaration

- The first character of a variable name can be only a-z, A-Z, $, or _ .

-- Then followed by only the letters a-z, A-Z, 0-9, the $ symbol, and the underscore _.

- Variable names are case-sensitive. Count, count and COUNT are three different variables
- Variable can be declared using
  - let (block scope)
  - var (function or global scope)
  - Const(block scope)
  - use without declaring (global scope)

KEYWORD	SCOPE	CAN BE REASSIGNED	CAN BE REDECLARED
var	Function	Yes	Yes
let	block	Yes	No
const	block	No	No

JavaScript - Basics
JavaScript Code – Datatypes

```
function example() {
  if (true) {
    var varVariable = "I'm a var"; // Function-scoped
    let letVariable = "I'm a let"; // Block-scoped
  }

  console.log(varVariable); // Outputs "I'm a var"
  console.log(letVariable); // Throws ReferenceError
}

example();

// Re-declaration example
var redeclareVar = "Original var";
var redeclareVar = "Re-declared var"; // No error

let redeclareLet = "Original let";
// let redeclareLet = "Attempt to redeclare let"; // SyntaxError
```

JavaScript - Basics
JavaScript Code – Datatypes

- JS is loosely typed or dynamic typed
- Primitive Datatypes
  - number
  - string
  - boolean
  - null
  - undefined
- Non-Primitive Datatypes (used with new keyword)
  - Object
  - Number
  - String
  - Date
  - Array
  - Boolean

JavaScript – Objects

- In JavaScript, almost "everything" is an object.

- Booleans can be objects (if defined with the new keyword)
- Numbers can be objects (if defined with the new keyword)
- Strings can be objects (if defined with the new keyword)
- Dates are always objects
- Math are always objects
- Regular expressions are always objects
- Arrays are always objects
- Functions are always objects
- Objects are always objects
- All JavaScript values, except primitives, are objects.

JavaScript - Basics
JavaScript Code – Operators and Constructs

JavaScript has most of the operators we're used to from C/Java

- Arithmetic (+, - , *, /, %) .
- Assignment (=, +=, -=, *=/=, %=, ++, --)
- Logical (&&, ||, !)
- Comparison (<, >, <=, >=, ==, ===,!=,!==)

Note: + also does concatenation if one of the operands is string

- Constructs: if, else, while, for, switch, case

JavaScript - Basics
JavaScript Code – Data types

```
let length = 16;                    // Number
let lastName = "Johnson";           // String
let x = 16 + "Volvo";
```

Output: 16Volve

When adding a number and a string, JavaScript will treat the number as a string.

```
let x = "Volvo" + 16;
```

Output: Volve16

```
let x = 16 + 4 + "Volvo";
```

Output: 20Volve

```
let x = "Volvo" + 16 + 4;
```

Output: Volve164

- In the first example, JavaScript treats 16 and 4 as numbers, until it reaches "Volvo".
- In the second example, since the first operand is a string, all operands are treated as strings.

JavaScript - Basics
JS concatenation and Addition (+)

- The + operator can also be used to add (concatenate) strings.
- var txt1 = ”A";
  var txt2 = ”SECTION";
  var txt3 = txt1 + " " + txt2; // A SECTION
- var txt1 = "What a very ";
  txt1 += "nice day"; // What a very nice day
- var x = 5 + 5; // 10
  var y = "5" + 5; //55
  var z = "Hello" + 5; // Hello5

JavaScript - Basics
Loops

- JavaScript supports different kinds of loops:

1. **for** - loops through a block of code a number of times
2. **for/in** - loops through the properties of an object
3. **while** - loops through a block of code while a specified condition is true
4. **do/while** - also loops through a block of code while a specified condition is true

Equality

```
1  console.log(1 == 1);
2  // expected output: true
3
4  console.log('hello' == 'hello');
5  // expected output: true
6
7  console.log('1' ==  1);
8  // expected output: true
9
10 console.log(0 == false);
11 // expected output: true
12
```


Strict Equality

```
1  console.log(1 === 1);
2  // expected output: true
3
4  console.log('hello' === 'hello');
5  // expected output: true
6
7  console.log('1' ===  1);
8  // expected output: false
9
10 console.log(0 === false);
11 // expected output: false
```

JavaScript - Functions

- A function is a subprogram designed to perform a particular task. Functions are executed when they are called. This is known as invoking a function.
- Functions always return a value. In JavaScript, if no return value is specified, the function will return undefined.
- Whenever you have a relatively complex piece of code that is likely to be reused, you have a candidate for a function.
- A **Function Declaration** defines a named function. To create a function declaration you use the function keyword followed by the name of the function. When using function declarations, the function definition is hoisted, thus allowing the function to be used before it is defined.

```
function name(parameters){
        statements
        }
```

JavaScript - Functions

- A **Function Expression** defines a named or anonymous function. An anonymous function is a function that has no name. Function Expressions are not hoisted, and therefore cannot be used before they are defined.
- In the example below, setting the anonymous function object equal to a variable.

```
let name = function(parameters){
        statements
        }
```

- An **Arrow Function Expression** is a shorter syntax for writing function expressions.

```
let name = (parameters) => {
        statements
        }
```

JavaScript - Functions
What are Functions?

- Whenever you have a relatively complex piece of code that is likely to be reused, you have a candidate for a function.
- The general syntax for a function is:

```
function function_name([parameter [, ...]])
{
        statements
        //optional return statement
}
```

- The general syntax for calling a function is:

```
[retval =] function_name([argument [,...]])
```

JavaScript - Functions
Parameters and Arguments

- Parameters are used when defining a function, they are the names created in the function definition. Parameters are separated by commas in the ().
- Arguments, on the other hand, are the values the function receives from each parameter when the function is executed (invoked).
- Argument list and parameter list mismatch does not give errors.
- Parameter that is not passed a value in arguments list is treated as **undefined**
- To access additional arguments, use the **arguments** array to access the values passed.

JavaScript - Functions
Hoisting – Variables and Functions

- Hoisting is JavaScript's default behavior of moving all variable and function declarations to the top of the current scope (to the top of the current <script> or the current function).
- Only declarations are hoisting not initializations

```
num = 4;
console.log(num);
var num = 9;
console.log(num);
```

⇨

```
var num;
num = 4;
console.log(num);
num = 9;
console.log(num);
```

JavaScript - Functions
Hoisting – Variables and Functions

- In JavaScript, a variable can be declared after it has been used.
- In other words; a variable can be used before it has been declared.

```
Example1
x = 5; // Assign 5 to x
document.write("Value of x "+x);  // Display x
var x; // Declare x*/

Example2
var x; // Declare x
 x = 5; // Assign 5 to
 document.write("Value of x "+x);
```

Both example 1 and example are same

JavaScript - Functions
Hoisting

- If we attempt to use a variable before it has been declared and initialized, it will return undefined.

Example:

```
console.log(x);
var x = 100;
```

⇨

```
// How JavaScript interpreted it
Var x;
Console.log(x);
x=100;
```

Output:
undefined

- However, if we omit the var keyword, we are no longer declaring the variable, only initializing it. It will return a ReferenceError and halt the execution of the script.

```
console.log(x);
 x = 100;
```

Output:
ReferenceError: x is not defined

The reason for this is due to hoisting. Since only the actual declaration is hoisted, not the initialization, the value in the first example returns undefined.

JavaScript - Functions
Hoisting (continued)

- Function declarations are fully hoisted. This means you can call a function before its declaration in the code, and it will work as expected.

```
sayHello();
        function sayHello() {
        console.log("Hello!");
       }
```

- Output: Hello!
- Function declarations are fully hoisted, while variable declarations are hoisted with an initial value of `undefined`

- Hoisting can lead to unpredictable results.

Example:

```
var a = 10;
function hoist() {
if (false) {
        var a = 20;
        }
console.log(a);                    Output:
}                                  undefined
hoist();
```

- In this example, we declared a to be 10 globally. Depending on an if statement, a could change to 20, but since the condition was false it should not have affected the value of a. Instead, a was hoisted to the top of the hoist() function, and the value became undefined.

JavaScript - Functions
Hoisting (continued)

- Variables and constants declared with let or const are not hoisted as they are block-scoped.
- Duplicate declaration of variables, which is possible with var, will throw an error with let and const.
- // Attempt to overwrite a variable declared with var

```
var a = 3;
var a = 4;
console.log(a);      Output: 4
```

- // Attempt to overwrite a variable declared with let

```
let y = 1;
let y = 2;
console.log(y);      Output: Uncaught SyntaxError: Identifier 'y' has already been declared.
```

JavaScript – Built-in Objects
Global Object

- When the JavaScript interpreter starts up, there is always a single global object instance created. All other objects are properties of this object.
- Also, all other variables and functions are properties of this global object.
- For client-side JavaScript, this object is the instance **window** of the constructor Window.
- . Since everything is a property of the window object, there is a relaxation of the dot notation when referring to properties. Thus, each reference to a variable, function or object is not required to start with "window."
- Both these lines have the same effect:

```
window.status = "this is a test";
status = "this is a test";
```

JavaScript – Built-in Objects
Global Objects supported

- Math
- Number
- String
- Array
- Date
- window
- document

JavaScript – Built-in Objects
Array

- Arrays are lists of elements indexed by a numerical value starting with 0 to length - 1
- Arrays can be created using
  - The new Array method
    - let arr = new Array(100) – creates an array of 100 elements
    - let arr = new Array(10, 20) – creates an array of 2 elements
  - Literal arrays using square brackets
    - var alist = [1, "ii", "gamma", "4"];

JavaScript – Built-in Objects
Array Length

- Array length property can be modified at runtime
- Hence, the length property does not necessarily indicate the number of defined values in the array

```
const arr = [1, 2];
console.log(arr);
// [ 1, 2 ]

arr.length = 7; // set array length to 5 while currently 2.
console.log(arr);
// [ 1, 2, <5 empty items> ]

for (i in arr)
   console.log(typeof i + i));
// string 0
// string 1
```

Array Elements Can Be Objects

JavaScript variables can be objects. Arrays are special kinds of objects. We can have objects in an Array. We can have functions in an Array. We can have arrays in an Array.

myArray[0] = Date.now;
myArray[1] = myFunction;
myArray[2] = myCars;

- push – Add to the end
- pop – Remove from the end
- shift – Remove from the front
- unshift – Add to the front
- join – return a string with array elements
- indexOf – return the index of array
- sort – sort an array in ascending order by default
- concat – concatenate two arrays
- slice – returns a subset if of the array

Adding Array Elements

Both methods adds a new element (WT) to Courses

var courses=['HTML' , 'CSS', 'Javascript', 'React']
courses.push('WT')
Or
courses[Courses.length] = "WT"

1) Converting Arrays to Strings
const array1 = [1, 2, 'a', '1a'];
console.log(array1.toString()); // expected output: "1,2,a,1a"
2) The join() method also joins all array elements into a string.
const elements = ['Fire', 'Air', 'Water'];
console.log(elements.join());// expected output: "Fire,Air,Water"

3) **Popping and Pushing**

The pop() method removes the last element from an array:

Eg1:

```
var names = ['Blaire', 'Ash', 'Coco', 'Dean', 'Georgia'];
var remove = names.pop();
console.log(names);
console.log(remove);
▸(4) ["Blaire", "Ash", "Coco", "Dean"]
Georgia
```

Eg 2:

```
> var array = [];
array.push(10, 20, 30, 40, 50, 60, 70);
console.log(array)
▸(7) [10, 20, 30, 40, 50, 60, 70]
```

4) Shifting Elements

The shift() method removes the first array element and "shifts" all other elements to a lower index. Example:

```
var names = ['Joey','Rachel','Monica']
var initialEle = names.shift()
console.log(names)
console.log(initialEle)
[ 'Rachel', 'Monica' ]
Joey
```

The **unshift()** method adds a new element to an array at the beginning

Example:

```
var names = ['Joey','Rachel','Monica']
names.unshift('Phoebe','Ross')
console.log(names);
[ 'Phoebe', 'Ross', 'Joey', 'Rachel', 'Monica' ]
```


5) Deleting Elements

Eg :

```
let arr = [1,2,3,4,5]
 delete arr[2]
 console.log(arr)
 Array(5) [ 1, 2, <1 empty slot>, 4, 5 ]  //replaces with "undefined"
```

6) Splicing an Array

The splice() method changes the contents of an array by removing or replacing existing elements and/or adding new elements in place

Eg:

```
arr=[1,2,3,4,5]
 let rm = arr.splice(2,1) // remove 1 element from index 2
 console.log(rm)
 Array [ 3 ] // returns array with removed elements
```


- The first parameter (2) defines the position where new elements should be added (spliced in).
- The second parameter (2) defines how many elements should be removed.
- The rest of the parameters ("Node", "Express") define the new elements to be added.
- The splice() method returns an array with the deleted items:

Eg –

var Courses = ["HTML", "CSS", "JavaScript", "React"];
Courses.splice(2, 2, "Node", "Express");

7) Merging (Concatenating) Arrays

The concat() method creates a new array by merging (concatenating) existing arrays

var arr1 = ["Cecilie", "Lone"];
var myChildren = arr1.concat(["Emil", "Tobias", "Linus"]);


```
const months = ['Jan', 'March', 'April', 'June'];
months.splice(1, 0, 'Feb');
// inserts at index 1
console.log(months);
// expected output: Array ["Jan", "Feb", "March", "April", "June"]

months.splice(4, 1, 'May');
// replaces 1 element at index 4
console.log(months);
// expected output: Array ["Jan", "Feb", "March", "April", "May"]
```

8) **Sorting an Array**

The sort() method sorts an array alphabetically.

```
var fruits = ["Banana", "Orange", "Apple", "Mango"];
fruits.sort();        // Sorts the elements of fruits
```

9) The **reverse()** method reverses the elements in an array.

```
fruits.reverse();
```


- *arr*.sort(*[compareFunction]*)
- If compareFunction is not specified,array is sorted as strings in ascending order
- Arr = [1, 2, 11, 12, 22]
- Console.log(arr.sort())
- // 1, 11, 12, 2, 22

```
var numbers = [4, 2, 5, 1, 3];
numbers.sort(function(a, b) { return b - a; });
console.log(numbers);
// [5,4,3,2,1]
```

- compareFunction takes two parameters say a and b and returns
  - 1 if a > b
  - 0 if a = b
  - -1 if a < b
- To reverse the order modify the condition for returning 1, 0 and -1

Array.sort- Compare function

- **compareFunction(Optional):** A function that defines an alternative sort order. The function should return a negative, zero, or positive value, depending on the arguments, like:

  function(a, b)

  {return a-b}
- When the sort() method compares two values, it sends the values to the compare function, and sorts the values according to the returned (negative, zero, positive) value.
- Example:
- When comparing 40 and 100, the sort() method calls the compare function(40,100).
- The function calculates 40-100, and returns -60 (a negative value).
- The sort function will sort 40 as a value lower than 100.

- MAX_VALUE : represents the maximum numeric value representable in JavaScript.
- MIN_VALUE
- NaN – Not a Number
- POSITIVE_INFINITY
- NEGATIVE_INFINITY

```
const biggestNum     = Number.MAX_VALUE
const smallestNum    = Number.MIN_VALUE
const infiniteNum    = Number.POSITIVE_INFINITY
const negInfiniteNum = Number.NEGATIVE_INFINITY
const notANum        = Number.NaN
```

- Operations resulting in errors return NaN
  - Use isNaN(a) to test if a is NaN
- toString method converts a number to string

JavaScript – Built-in Objects

NaN- Not a Number

```
function sanitise(x) {
  if (isNaN(x)) {
    return NaN;
  }
  return x;
}

console.log(sanitise('1'));
// expected output: "1"

console.log(sanitise('NotANumber'));
// expected output: NaN
```

JavaScript – Built-in Objects

String Object

Method	Description
charAt(*index*)	Returns a string containing the character at the specified *index*. If there is no character at the *index*, charAt returns an empty string. The first character is located at *index* 0.
charCodeAt(*index*)	Returns the Unicode value of the character at the specified *index*. If there is no character at the *index*, charCodeAt returns NaN (Not a Number).
concat(*string*)	Concatenates its argument to the end of the string that invokes the method. The string invoking this method is not modified; instead a new String is returned. This method is the same as adding two strings with the string concatenation operator + (e.g., s1.concat(s2) is the same as s1 + s2).
fromCharCode(*value1, value2, ...*)	Converts a list of Unicode values into a string containing the corresponding characters.
indexOf(*substring, index*)	Searches for the first occurrence of *substring* starting from position *index* in the string that invokes the method. The method returns the starting index of *substring* in the source string or –1 if *substring* is not found. If the *index* argument is not provided, the method begins searching from index 0 in the source string.
lastIndexOf(*substring, index*)	Searches for the last occurrence of *substring* starting from position *index* and searching toward the beginning of the string that invokes the method. The method returns the starting index of *substring* in the source string or –1 if *substring* is not found. If the *index* argument is not provided, the method begins searching from the end of the source string.

JavaScript – Built-in Objects

String Object

Method	Description
slice(*start, end*)	Returns a string containing the portion of the string from index *start* through index *end*. If the *end* index is not specified, the method returns a string from the *start* index to the end of the source string. A negative *end* index specifies an offset from the end of the string starting from a position one past the end of the last character (so –1 indicates the last character position in the string).
split(*string*)	Splits the source string into an array of strings (tokens) where its *string* argument specifies the delimiter (i.e., the characters that indicate the end of each token in the source string).
substr (*start, length*)	Returns a string containing *length* characters starting from index *start* in the source string. If *length* is not specified, a string containing characters from *start* to the end of the source string is returned.
substring(*start, end*)	Returns a string containing the characters from index *start* up to but not including index *end* in the source string.
toLowerCase()	Returns a string in which all uppercase letters are converted to lowercase letters. Non-letter characters are not changed.
toUpperCase()	Returns a string in which all lowercase letters are converted to uppercase letters. Non-letter characters are not changed.
toString()	Returns the same string as the source string.
valueOf()	Returns the same string as the source string.

JavaScript – Built-in Objects

Date Methods

var now= new Date();

- This creates a Date object for the time at which it was created (stored as the number of milliseconds since January 1, 1970, UTC - Epoch)

toLocaleString	A string of the Date information
(get/set)Date	The day of the month
(get/set)Month	The month in the range of 0 to 11
(get/set)Day	The day of the week in the range of 0 to 6
(get/set)FullYear	The year
(get/set)Time	The number of milliseconds since January 1, 1970
(get/set)Hours	The number of the hour in the range of 0 to 23
(get/set)Minutes	The number of the minute in the range of 0 to 59
(get/set)Seconds	The number of the second in the range of 0 to 59
(get/set)Milliseconds	The number of the millisecond in the range of 0 to 999

JavaScript – Built-in Objects
Math Methods

Provides **static** mathematical constants and functions

Math.E	Euler's Constant (Approx. 2.718)
Math.PI	Value of PI (Approx. 3.1416)
Math.SQRT2	Square root of 2 (Approx. 1.414)
Math.abs(x)	Returns the absolute value of x
Math.ceil(x)	Returns the smallest integer greater than or equal to x
Math.floor(x)	Returns the largest integer less than or equal to x.
Math.max([x[, y[, …]]])	Returns the largest of zero or more numbers.
Math.min([x[, y[, …]]])	Returns the smallest of zero or more numbers.
Math.pow(x, y)	Returns base x to the exponent power y (that is, x^y).
Math.random()	Returns a pseudo-random number between 0 and 1.

JavaScript – Built-in Objects
Window – Properties and Methods

- Global object containing global variables and functions declared in the page. For example, *var x;* can also be accessed as *window.x*

location	object containing location details like href, path etc.
history	object containing the browser history
localStorage	object containing a local cache for storing user info
innerHeight, innerWidth	dimensions of the display area of the browser
alert(text)	method to display a dialog box with message
prompt(text,default)	method to seek input from user, returns string
confirm(text)	method to show a confirmation dialog
setInterval, clearInterval	start/stop performing action repeatedly after an interval
setTimeout, clearTimeout	start/stop performing action once after a timeout period

JavaScript – Built-in Objects
setTimeout and setInterval

- **setTimeout** allows us to run a function once after the interval of time.
- The syntax: let timerId = setTimeout(func|code, [delay], [arg1], [arg2], ...) where delay is in milliseconds

Example:
```
function sayHi(phrase, who) {
   alert( phrase + ', ' + who );
   }
   setTimeout(sayHi, 1000, "Hello", "John"); // Hello, John
```

- **setInterval** allows us to run a function repeatedly, starting after the interval of time, then repeating continuously at that interval.
- The setInterval method has the same syntax as setTimeout:
  let timerId = setInterval(func|code, [delay], [arg1], [arg2], ...)
- Example:
```
// repeat with the interval of 2 seconds
let timerId = setInterval(() => alert('tick'), 2000);
// after 5 seconds stop
setTimeout(() => { clearInterval(timerId); alert('stop'); }, 5000);
```

Javascript Objects
Basic Object Oriented Concepts

- An object in JavaScript is a reference data type.
- An object can be compared to any real world entities.
- An object is an unordered list of properties consisting of a name (always a string) and a value. When the value of a property is a function, it is called a method.

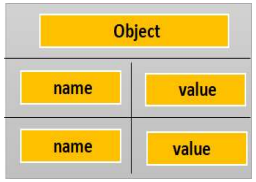


Fig 1:Object Structure

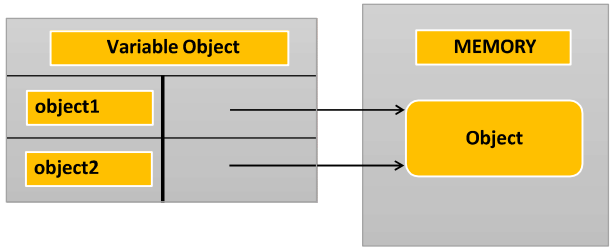


Fig 2:Object Reference

```
Var object1= new Object();
Var object2 =object1;
```

Javascript Objects
Object Literals using {key : value}

```
var emp2 = {
    "firstName": "Aruna",
    "lastName": "Srinivasan",
    "showName": function () {
        alert(this.firstName + " " + this.lastName);
    }
};
emp2.showName();
```

- Avariant of object literal syntax
- Specifies object members and their values inside the curly brackets as key-value pairs
- Amember and its value are delimited using colon (:) character.

Javascript Objects
Creating an object using function Constructor

```
function Employee(fname,lname) {
    this.firstName = fname;
    this.lastName = lname;
    this.showName = function () {
        alert(this.firstName + " " + this.lastName);
    };
}
var emp5 = new Employee("Aruna", "Srinivasan");
emp5.showName();
```

- JavaScript functions are objects themselves. It can be called as a Constructor Function.
- Once the function constructor is created you create object of that function using new keyword.
- The function name uses camel case convention

Javascript Objects
Creating an object using anonymous function

```
var emp6 = new function () {
    this.firstName = "Aruna";
    this.lastName = "Srinivasan";
    this.showName = function () {
        alert(this.firstName + " " + this.lastName);
    };
}
emp6.showName();
```

- Two steps are performed here
  - Firstly, it creates an anonymous constructor function
  - Secondly, it calls new on it to create its object.

Javascript Objects
Prototype property

- Prototype property of an object holds the structure of that object
- It is shared by all object instances created using that constructor
- This can be used to modify/add properties to all instances after they have been created
- Methods should be added to prototype since only one copy of the method is created. If methods are added to constructor, each object instance holds its own copy of the method.

Employee.prototype.showName = function(){…}

Javascript Objects
Prototype property

```
function Car(make, model, year, owner) {
  this.make = make;
  this.model = model;
  this.year = year;
  this.owner = owner;
}
```

To instantiate the new objects, you then use the following:

```
var car1 = new Car('Eagle', 'Talon TSi', 1993, rand);
var car2 = new Car('Nissan', '300ZX', 1992, ken);
```

Javascript Objects
Prototype property

Note that you can always add a property to a previously defined object. For example, the statement

```
car1.color = 'black';
```

JavaScript `prototype` property allows you to add new properties to object constructors:

```
Car.prototype.color = null;
car1.color = 'black';
```

Javascript Objects
Prototype property

```
function Person(first, last, age, eye) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
}

Person.prototype.name = function() {
  document.write(this.firstName + " " + this.lastName);
};

var p1 = new Person("John", "Doe", 50, "blue");
p1.name();
```

Javascript Objects

Creating an object using EcmaScript 6 Class Keyword

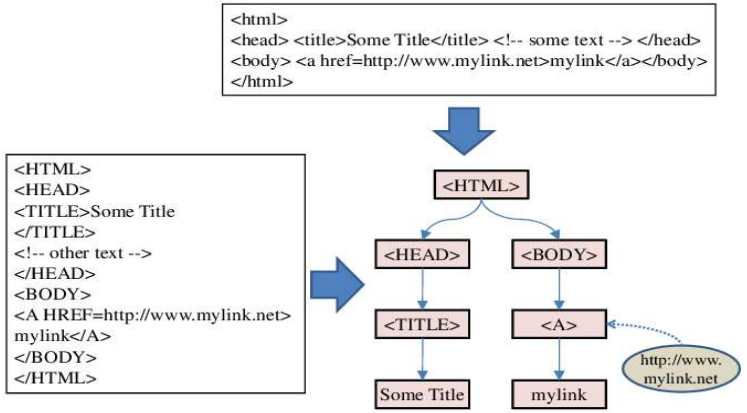
```
class Employee {
    constructor(fname, lname)
      {  this.firstName = fname;
        this.lastName = lname;   }
        showName()
        {   alert(this.firstName + " " + this.lastName); }
}
var emp7 = new Employee("Aruna", "Srinivasan");
emp7.showName();
emp7.firstName = "Sanvi";
emp7.showName();
```

- Class is defined .
- Then the objects of the class are created.
- It internally uses the constructor/prototype based approach of creating objects

HTML5,Jquery and Ajax
DOM – Document Object Model

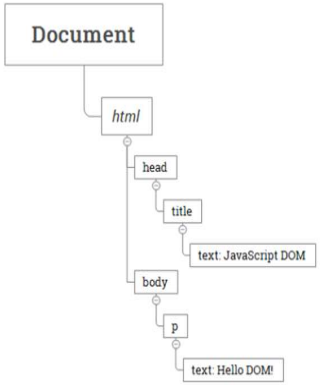
- When a web page is loaded, the browser creates a **D**ocument **O**bject **M**odel of the page.
- The **HTML DOM** model is constructed as a tree of **Objects.**
- The HTML DOM is a standard **object** model and **programming interface** for HTML. It defines:
  - The HTML elements as **objects**
  - The **properties** of all HTML elements
  - The **methods** to access all HTML elements
  - The **events** for all HTML elements

HTML5,Jquery and Ajax
DOM – Document Object Model



HTML5,Jquery and Ajax
DOM – Tree Representation

```
<html>
  <head>
    <title>JavaScript DOM</title>
  </head>
  <body>
    <p>Hello DOM!</p>
  </body>
</html>
```



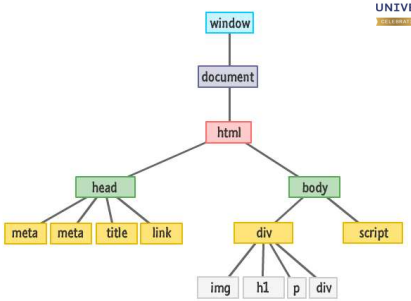
In this DOM tree, the document is the root node. The root node has one child which is the <html> element. The <html> element is called the document element.

Document Object Model
Drawbacks of using document.write()

- *"The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document."*
- Document.write executed after the page has finished loading will overwrite the page, or write a new page, or not work
- Document.write practically only appending to the page

Document Object Model
Introduction to DOM

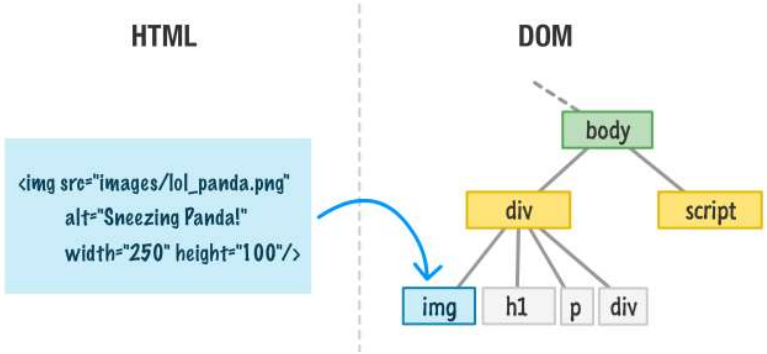
- A Web page is a document. This document can be either displayed in the browser window or as the HTML source. But it is the same document in both cases.
- The DOM is an object-oriented representation of the web page, which can be modified with a scripting language such as JavaScript.



DOM

- Objects have properties and methods, and respond to events.
  - Properties – specify attributes or characteristic of object .
  - Methods – specify functions object can perform.
  - Events – methods corresponding to user actions.

Document Object Model
DOM Elements are Objects



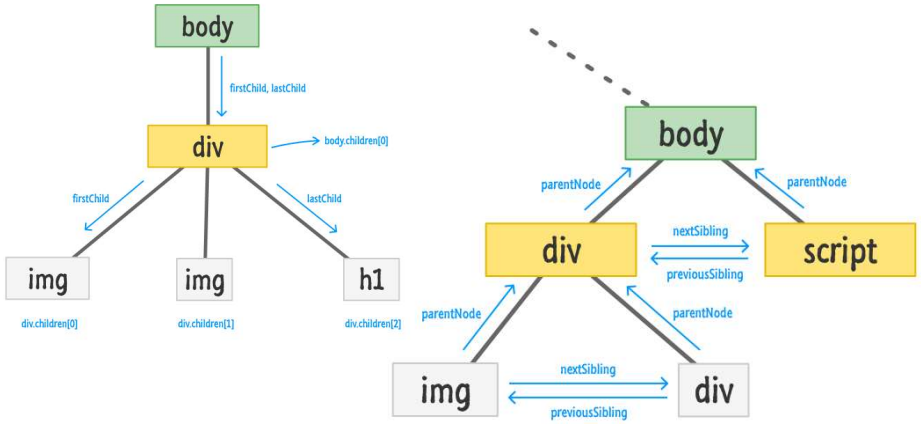
DOM
Accessing DOM

- **write("string"):** writes the given string on the document.
- **getElementById():** returns the element having the given id value.
- **getElementsByName():** returns all the elements having the given name value.
- **getElementsByTagName():** returns all the elements having the given tag name.

Document Object Model
Accessing Elements in DOM

Access Element By	Equivalent Selector	Method
ID	#demo	getElementByID(“demo”)
Class	.demo	getElementsByClassName(“demo”)
Tag	<tag name> like p	getElementsByTagName(“p”)
Selector (single)	Any CSS Selector	querySelector(“selector”)
Selector (all)		querySelectorAll(“selector”)

Document Object Model
Traversing the DOM



Document Object Model
Creating Element Objects

Method	Description
document.createElement()	Create a new element node using tag
document.createTextNode()	Create a new text node

Property	Description
node.textContent or node.innerText	Get or set the text content of an element node (without HTML tags)
node.innerHTML	Get or set the HTML content enclosed in the element tag

Document Object Model
Manipulating Nodes in the DOM

Method	Description
node.appendChild()	Add a node as the last child of the parent element.
node.insertBefore()	Insert a node into the parentbefore a specific sibling node
node.replaceChild()	Replace an existing node with a new node
node.removeChild()	Removes child node
node.remove()	Removes a node

* **node** here can be document.body or any existing element in the DOM

EVENT
What are Events?

EVENT
Events

Examples of HTML events:

- When a user clicks the mouse
- When a web page has loaded
- When an image has been loaded
- When the mouse moves over an element
- When an input field is changed
- When an HTML form is submitted
- When a user strokes a key

EVENT
A simple event handler Example

```
<form method="POST" action="">
    <input type="button"
           name="myButton"
           value="Click Me"
           onclick="alert('you clicked the button');">
</form>
```

EVENT
Event Handlers and Event Listeners

- Events are created by activities associated with specific HTML elements.
- The process of connecting an event handler to an event is called **registration**.
- There are two distinct approaches to event handler registration,
  - Assign element attributes
    - Inline event handlers
  - Assign handler addresses to object properties
    - Event handler properties and Event listeners

EVENT
Assigning events to elements

There are three ways to assign events to elements:

•Inline event handlers

•Event handler properties

•Add Event listeners

EVENT
Assigning events to elements

•Inline event handlers

```
<button onclick="bgChange()">Press me</button>
```

```
function bgChange() {
  const rndCol = 'rgb(' + random(255) + ',' + random(255) + ',' + random(255) + ')';
  document.body.style.backgroundColor = rndCol;
}
```

Event
Event Handler Properties

- An element can be assigned the event handler property

  element.on<event> = handler;

- It involves two parts
  - the correct event name it is to be listening for
  - the handler callback function.

For example:

div.onclick = change; or

div.onmouseover = function(){ … };

EVENT
Assigning events to elements

•Event handler properties

```
const btn = document.querySelector('button');

btn.onclick = function() {
  const rndCol = 'rgb(' + random(255) + ',' + random(255) + ',' + random(255) + ')';
  document.body.style.backgroundColor = rndCol;
}
```

Event
Event Listeners

- An event listener watches for an event on an element.

element.addEventListener(event, handler)

- It takes two mandatory parameters
  - the event it is to be listening for.
  - the handler callback function.

For Example:

div.addEventListener(“click”, change);

div.addEventListener(“keypress”, function(){ … });

EVENT
Assigning events to elements

•Event listeners

```
const btn = document.querySelector('button');

function bgChange() {
  const rndCol = 'rgb(' + random(255) + ',' + random(255) + ',' + random(255) + ')';
  document.body.style.backgroundColor = rndCol;
}

btn.addEventListener('click', bgChange);
```

```
btn.addEventListener('click', function() {
  var rndCol = 'rgb(' + random(255) + ',' + random(255) + ',' + random(255) + ')';
  document.body.style.backgroundColor = rndCol;
});
```

Events
Event Sources and example events

Source	Event	Fires When…
Mouse	click	the mouse is clicked and released on an element
	dblclick	an element is clicked twice
	mousemove	every time a mouse pointer moves inside an element
	mouseover	every time a mouse pointer is placed over an element
Keyboard	keydown	when a key is pressed down
	keyup	when a key pressed is released
	keypress	when a key is pressed and released
Form	submit	a form is submitted
	reset	a form reset button is clicked
	focus	an input element is clicked and receives focus
	blur	an input element loses focus

Event
Event Object Properties

- Event object holds the context or details of the event

Property	IE5-8 Equivalent	Specifies
target	srcElement	the target of the event (most specific element).
type	-	the name of event fired (without the on prefix)
altKey / shiftKey / ctrlKey / metaKey	-	true/false to signify if Alt Key or Shift Key or Ctrl Key or Meta Key was pressed
charCode	keyCode	Unicode character code of the pressed key
key	-	Key Character Name (‘a’ or ‘F1’ or ‘CAPS LOCK’)
button	-	Returns which mouse button was pressed
clientX, clientY / offsetX, offsetY / screenX, screenY	-	the coordinates of the mouse pointer when the event triggered, relative to, the current window / target element / screen

Event Handling
Event Propagation

Ways of event propagation in the HTML DOM,

- Event bubbling
- Event capturing.
- Target Phase

- **Propagation** ***is a mechanism that defines how events propagate or travel through the DOM tree to arrive at its target.***
- Event propagation is a way of defining the element order when an event occurs.

```
<div>
   <p>  /<p>
</div>
```

- If both <p> and elements registered Click event, which element's "click" event should be handled first?

Event Handling
Event Propagation

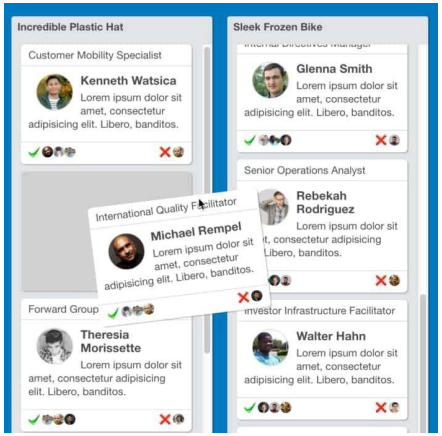
bubbling

- inner most element's event is handled first and then the outer: the <p> element's click event is handled first, then the element's click event.

capturing

- outer most element's event is handled first and then the inner: the element's click event will be handled first, then the <p> element's click event.

Event Handling
Event Propagation



Event Handling
Event Flow

There are three phases in which an event can propagate to handlers defined in parent elements:

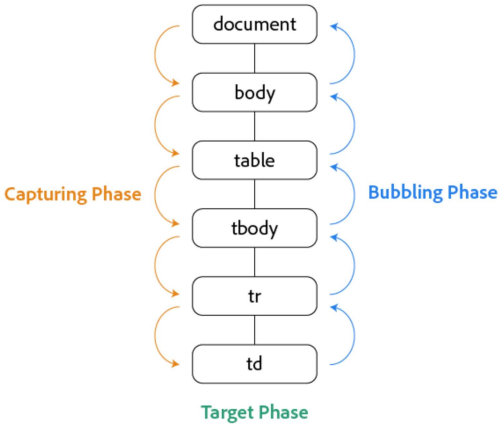
- Capturing phase
- Target phase
- Bubbling phase

elem.addEventListener("event", func_ref, **flag**);

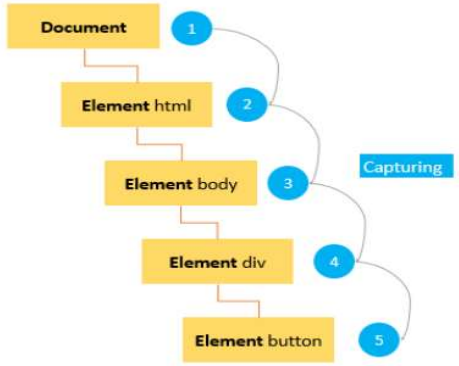
flag = true :=> Handler registered for Capturing phase

flag = false:=> Handler registered for Bubbling phase (default)

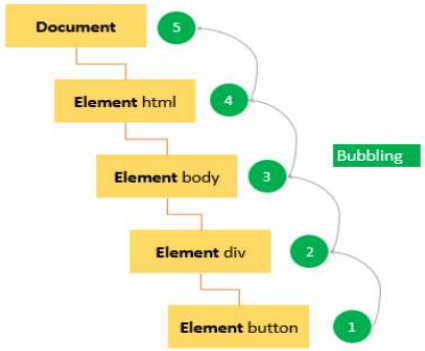
Event Flow
Event Capturing



Event Flow
Event Capturing



Event Flow
Event Bubbling



Event Flow
Event Capturing

The eventPhase event property returns a number that indicates which phase of the event flow is currently being evaluated.

The number is represented by 4 constants:

0	NONE	
1	CAPTURING_PHASE	The event flow is in capturing phase
2	AT_TARGET	The event flow is in target phase, i.e. it is being evaluated at the event target
3	BUBBLING_PHASE	The event flow is in bubbling phase

Event Handling
Event Object Properties

- Event object has properties and methods related to Bubbling and Capturing

Property/ Method	IE5-8 Equivalent	Purpose
cancelBubble	-	A historical alias to stopPropagation(). Setting its value to **true** before returning prevents propagation
eventPhase	-	Specifies which phase of the event flow is being processed
cancelable	Not supported	Indicates whether you can cancel the default behaviour of an element
preventDefault()	returnValue	It cancels the default behavior of the event (if possible)
stopPropogation()	cancelBubble	It stops any further bubbling/ capturing of the event.